



CSE 320 - Computer Networks B

LAB Session 4

13.04.2026

Simple TCP Communication Lab in C

TCP Calculator Lab

Goal: By the end of the lab, students should be able to:

1. create a TCP socket in C
2. bind and listen on a server
3. connect from a client
4. send and receive data over TCP
5. run both programs from the Windows command line

An example TCP system:

1. **server** waits for a connection
2. **client** connects and sends a message
3. **server** prints the message and replies
4. **client** prints the reply

Server:

- listens on 127.0.0.1:8082
- receives: "12 5"
- Computes:
 - sum = 17
 - difference = 7
- Sends:

SUM: 17 | DIFF: 7

Client:

- asks user for two integers
- sends them as a single string
- prints server reply

1. Prepare files

- **server.c**, **client.c**

2. Add required headers

server.c

```
1  #include <stdio.h>
2  #include <string.h>
3  #include <winsock2.h>
4
5  #pragma comment(lib, "ws2_32.lib")
6
```

client.c

```
1  #include <stdio.h>
2  #include <string.h>
3  #include <winsock2.h>
4
5  #pragma comment(lib, "ws2_32.lib")
6
```

You'll need c socket functions and window's own socket library.

3. Start WinSock

Both programs will require the following

```
WSADATA wsa;
WSAStartup(MAKEWORD(2,2), &wsa);
```

4. Create a TCP Socket

AF_INET IPv4, SOCK_STREAM TCP

```
SOCKET s = socket(AF_INET, SOCK_STREAM, 0);
```

5. Start main()

Create the main functions in both programs (client.c and server.c)

```
5 int main()  
6 {  
7  
8 }  
9
```

A. Server Side

1. Declare Variables

```
WSADATA wsa;  
SOCKET server_socket, client_socket;  
struct sockaddr_in server, client;  
int client_size;  
  
char buffer[1024] = {0};  
char reply[1024];  
  
int a, b;
```

2. Initialize Winsock

```
17  
18 WSStartup(MAKEWORD(2,2), &wsa);  
19
```

3. Create Server Socket and Set Server Address

In the server program

```

20     server_socket = socket(AF_INET, SOCK_STREAM, 0);
21
22     server.sin_family = AF_INET;
23     server.sin_addr.s_addr = inet_addr("127.0.0.1");
24     server.sin_port = htons(8082);
25

```

4. Server “Bind”

Bind operation that’s specific to the server model

```

26     bind(server_socket, (struct sockaddr *)&server, sizeof(server));

```

5. “Listen”

Operations to listen from server side

```

27
28     listen(server_socket, 1);
29     printf("Calculator Server running on port 8082...\n");
30

```

6. When client is sending a message, server “Accept”s it

```

31     client_size = sizeof(struct sockaddr_in);
32     client_socket = accept(server_socket,
33                           (struct sockaddr *)&client, &client_size);
34

```

7. “Receive” message from the Client

```

34
35     recv(client_socket, buffer, sizeof(buffer), 0);
36     printf("Received: %s\n", buffer);
37

```

8. Parse Input and Reply Client

```

37
38     // Parse input
39     sscanf(buffer, "%d %d", &a, &b);
40
41     // Compute
42     int sum = a + b;
43     int diff = a - b;
44
45     // Format reply
46     sprintf(reply, "SUM: %d | DIFF: %d", sum, diff);
47

```

```
47  
48     send(client_socket, reply, strlen(reply), 0);  
49
```

9. Close Existing Sockets for both Server listen and Message

```
54     closesocket(client_socket);  
55     closesocket(server_socket);  
56     WSACleanup();  
57  
58     return 0;
```

B. Client Side

1. Declare Variables

```
8         WSADATA wsa;  
9         SOCKET s;  
10        struct sockaddr_in server;  
11  
12        char message[1024];  
13        char server_reply[1024];  
14  
15        int a, b;  
16
```

2. Initialize Winsock

```
16  
17        WSAStartup(MAKEWORD(2,2), &wsa);  
18
```

3. Create Socket and Set Server Info

```
18  
19        s = socket(AF_INET, SOCK_STREAM, 0);  
20  
21        server.sin_family = AF_INET;  
22        server.sin_addr.s_addr = inet_addr("127.0.0.1");  
23        server.sin_port = htons(8082);  
24
```

4. Client "Connect"s

```
24  
25     connect(s, (struct sockaddr *)&server, sizeof(server));  
26
```

5. Get message from user and Remove newline from **fgets**

Removal at the last line

```
27     printf("Enter two integers (e.g., 10 5): ");  
28     scanf("%d %d", &a, &b);  
29  
30     sprintf(message, "%d %d", a, b);
```

6. Send Message

```
32     send(s, message, strlen(message), 0);  
33
```

7. Receive reply and Print

```
34     recv(s, server_reply, sizeof(server_reply), 0);  
35  
36     printf("Server replied: %s\n", server_reply);  
37
```

8. Close Socket and terminate

```
46     closesocket(s);  
47     WSACleanup();  
48  
49     return 0;
```

C. Compile and Run

Visual Studio (on **Developer Command Prompt for Visual Studio**)

cl server.c ws2_32.lib

cl client.c ws2_32.lib

or

Use MinGW gcc compile

<https://www.mingw-w64.org/downloads/#msys2>

<https://www.msys2.org/>

In the program file, run the mingw64.exe and type "pacman -S mingw-w64-x86_64-gcc" and type Y for installation

Locate to the code files. E.g. cd /c/Users/manol/lab4 (instead of C:\, /c/..)

```
gcc server.c -o server -lws2_32
```

```
gcc client.c -o client -lws2_32
```

Open Two terminal windows and run each program with their names

Terminal 1 -> server

Terminal 2 -> client

For MinGW

```
manol@Manolia_PC MINGW64 /c/Users/manol
$ ./server.exe
Calculator Server running on port 8082.
Received: 17 3
```

```
manol@Manolia_PC MINGW64 /c/users/m
$ ./client.exe
Enter two integers (e.g., 10 5): 17
3
Server replied: SUM: 20 | DIFF: 14
```